```
!pip install shap
```

```
Collecting shap
  Downloading shap-0.46.0-cp310-cp310-manylinux_2_12_x86_64.manylinux2010_x86_64.many
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from
Requirement already satisfied: scipy in /usr/local/lib/python3.10/dist-packages (from
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.10/dist-package
Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (fro
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.10/dist-package
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.10/dist-packa
Collecting slicer==0.0.8 (from shap)
  Downloading slicer-0.0.8-py3-none-any.whl.metadata (4.0 kB)
Requirement already satisfied: numba in /usr/local/lib/python3.10/dist-packages (from
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.1
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/di
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-package
Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.10/dist-packa
Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.10/dist-packag
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.10/dist
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (f
Downloading shap-0.46.0-cp310-cp310-manylinux_2_12_x86_64.manylinux2010_x86_64.manyli
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 540.1/540.1 kB 5.3 MB/s eta 0:00:00
Downloading slicer-0.0.8-py3-none-any.whl (15 kB)
Installing collected packages: slicer, shap
Successfully installed shap-0.46.0 slicer-0.0.8
```

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split

import shap
import numpy as np
import pandas as pd
import seaborn as sns
import statsmodels.api as sm
from collections import Counter
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, ElasticNet, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

from sklearn.svm import SVC, LinearSVC
from sklearn.metrics import accuracy_score
from tensorflow.keras.models import Sequential
from sklearn.neural_network import MLPClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.inspection import permutation_importance
```

```
from tensorflow.keras.layers import Dense, Conv1D, Flatten, MaxPooling1D
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_sco
```

Import and perform some data preporcessing

```
class DataBase:
    def __init__(self):
        self.__load_data()

    def __load_data(self):
        self.WaterQuality = pd.read_csv('waterQuality.csv') # Import your DATA <=====
        self.X = self.WaterQuality.iloc[:, :-1]
        self.y = self.WaterQuality.iloc[:, -1]
        self.co = 0
        self.cm = 0

    def check_missing_values(self, remove=False):
        if remove:
            missing_indices = self.X[self.X.isna().any(axis=1)].index
            self.X.drop(index=missing_indices, inplace=True)
            self.y.drop(index=missing_indices, inplace=True)
            self.X.reset_index(drop=True, inplace=True)
            self.y.reset_index(drop=True, inplace=True)
        else:
            missing_data = self.X.isna().sum()
            if missing_data.any():
                print('\nMissing data:')
                print(missing_data[missing_data > 0])
            else:
                print('\nNo missing data in dataset')

    def check_duplicated_rows(self, remove=False):
        if remove:
            self.X.drop_duplicates(keep=False, inplace=True)
            #print(f'Duplicate rows removed from {name}')
        else:
            duplicates = self.X.duplicated(keep=False)
            if duplicates.any():
                print(f'\nThere are {duplicates.sum()} duplicate rows in dataset at indic
                print(self.X[duplicates].index.tolist())
            else:
                print('\nNo duplicate rows in dataset')

    def check_duplicated_columns(self, remove=False):
        if remove:
            self.X = self.X.T.drop_duplicates(keep='first')
            self.X = self.X.T
            #print(f'Duplicates columns removed from {name}')
        else:
            duplicates = self.X.T.duplicated(keep=False).T
```

```
        if duplicates.any():
            print(f'\nThere are {duplicates.sum()} duplicate rows in dataset at indic
            print(self.X[duplicates].index.tolist())
        else:
            print('\nNo duplicate columns in dataset')

    def histbox_plot(self, col_index = 0, bins = 'auto'):

        self.check_missing_values(True)
        self.check_duplicated_rows(True)
        self.check_duplicated_columns(True)

        if self.co == 0:
            self.__remove_outliers()

        fig, axes = plt.subplots(1, 2)

        sns.histplot(self.X.iloc[:, col_index], bins=bins, ax=axes[0])
        axes[0].set_title('Histogram of ' + self.X.columns[col_index])

        sns.boxenplot(y=self.X.iloc[:, col_index], ax=axes[1])
        axes[1].set_title('Boxenplot of ' + self.X.columns[col_index])
        plt.show()

    def var_relation_plot(self):

        self.check_missing_values(True)
        self.check_duplicated_rows(True)
        self.check_duplicated_columns(True)

        corr = self.X.corr()
        plt.figure(figsize=(12, 8))
        sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm')
        plt.title('Correlation Heatmap for Numerical Features')
        plt.xticks(rotation=45, ha='right')
        plt.tight_layout()
        plt.show()

    def __remove_outliers(self):
        Q1 = self.X.quantile(0.25)
        Q3 = self.X.quantile(0.75)
        IQR = Q3 - Q1
        lb = Q1 - 1.5 * IQR
        ub = Q1 + 1.5 * IQR
        self.X = self.X.clip(lower=lb, upper=ub, axis=1)
        #print('\nOutliers are removed!')
        self.co = 1

    def prep_data(self, outlier = False, scale = False):
        self.check_missing_values(True)
        self.check_duplicated_rows(True)
        self.check_duplicated_columns(True)

        if outlier and (self.co == 0):
            self.__remove_outliers()
```

```
        if scale and (self.cm == 0):
            scaler = MinMaxScaler()
            self.X = pd.DataFrame(scaler.fit_transform(self.X), columns=self.X.columns)
            self.cm = 1
            #print('\nMin and Max scaling is applied!')

        Features = self.X
        Targets = self.y.str.lower().map({'yes': 1, 'no': 0, 'unknown': 0})
        Targets.values.ravel()

        return Features, Targets

    def TrainTest(self, X, y, tsize = 0.3):

        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=tsize, random

        return X_train, X_test, y_train, y_test
```

Define the important features

```
class ImpFeature:
    def __init__(self, X, y):
        self.X = X
        self.y = y.values.ravel()
        self.vote = []
        self.accuracies = {}
        self.__run_all()

    def __rf(self):
        rf = RandomForestClassifier(random_state=42)
        rf.fit(self.X, self.y)
        importances = rf.feature_importances_
        feature_names = self.X.columns
        feature_importances = pd.DataFrame({'feature': feature_names, 'importance': impor
        feature_importances = feature_importances.sort_values(by='importance', ascending=
        self.vote.append(feature_importances.index.values)
        self.accuracies['Random Forest'] = accuracy_score(self.y, rf.predict(self.X))

    def __svm(self):
        svm = LinearSVC(random_state=42, dual=True)
        svm.fit(self.X, self.y)
        importances = abs(svm.coef_).mean(axis=0)
        feature_names = self.X.columns
        feature_importances = pd.DataFrame({'feature': feature_names, 'importance': impor
        feature_importances = feature_importances.sort_values(by='importance', ascending=
        self.vote.append(feature_importances.index.values)
        self.accuracies['Linear SVM'] = accuracy_score(self.y, svm.predict(self.X))


    def __gb(self):
        gb = GradientBoostingClassifier(random_state=42)
        gb.fit(self.X, self.y)
        importances = gb.feature_importances_
```

```
        feature_names = self.X.columns
        feature_importances = pd.DataFrame({'feature': feature_names, 'importance': impor
        feature_importances = feature_importances.sort_values(by='importance', ascending=
        self.vote.append(feature_importances.index.values)
        self.accuracies['Gradient Boosting'] = accuracy_score(self.y, gb.predict(self.X))

    def __lr(self):
        lr = LogisticRegression(solver='lbfgs', random_state=42)
        lr.fit(self.X, self.y)
        importances = abs(lr.coef_).mean(axis=0)
        feature_names = self.X.columns
        feature_importances = pd.DataFrame({'feature': feature_names, 'importance': impor
        feature_importances = feature_importances.sort_values(by='importance', ascending=
        self.vote.append(feature_importances.index.values)
        self.accuracies['Logistic Regression'] = accuracy_score(self.y, lr.predict(self.X

    def __knn(self):
        knn = KNeighborsClassifier()
        knn.fit(self.X.values, self.y)
        result = permutation_importance(knn, self.X.values, self.y, n_repeats=20, random_
        importances = result.importances_mean
        feature_names = self.X.columns
        feature_importances = pd.DataFrame({'feature': feature_names, 'importance': impor
        feature_importances = feature_importances.sort_values(by='importance', ascending=
        self.vote.append(feature_importances.index.values)
        self.accuracies['KNN'] = accuracy_score(self.y, knn.predict(self.X.values))

    def __mlp(self):
        mlp = MLPClassifier(random_state=42)
        mlp.fit(self.X.values, self.y)
        result = permutation_importance(mlp, self.X.values, self.y, n_repeats=20, random_
        importances = result.importances_mean
        feature_names = self.X.columns
        feature_importances = pd.DataFrame({'feature': feature_names, 'importance': impor
        feature_importances = feature_importances.sort_values(by='importance', ascending=
        self.vote.append(feature_importances.index.values)
        self.accuracies['MLP'] = accuracy_score(self.y, mlp.predict(self.X))

    def find_repetitive_ind(self):
        repetitive_ind = []
        feature_names = []
        xin = np.vstack(self.vote).T
        for row in xin:
            counter = Counter(row)
            row_repeats = [item for item, count in counter.items() if count > 1]
            repetitive_ind.append(row_repeats)
            feature_names.append([self.X.columns[item] for item in row_repeats])

        for i in range(len(feature_names)):
            print(f'rank {i+1}: {feature_names[i]}')

    def __run_all(self):
        self.__rf()
        self.__svm()
        self.__gb()
```

```
        self.__lr()
        self.__knn()
        self.__mlp()

    def shap_analysis(self):
        svm = LinearSVC(random_state=42, dual=False)
        svm.fit(self.X, self.y)

        small_X = self.X.sample(100, random_state=42)  # Random sample of 100 rows

        explainer = shap.KernelExplainer(svm.predict, small_X)
        shap_values = explainer.shap_values(self.X)

        shap.summary_plot(shap_values, self.X, feature_names=self.X.columns)
```

Build Classifiers

```
class NeuralNetWorkModels:
    def __init__(self, X_train, X_test, y_train, y_test, epochs = 100, batch_size = 5, va

        self.X_train, self.X_test, self.y_train, self.y_test = X_train.values, X_test.val
        self.val_split = val_split
        self.batch_size = batch_size
        self.epochs = epochs
        self.h = h

        self.X_train_cnn = self.X_train.reshape((self.X_train.shape[0], self.X_train.shap
        self.X_test_cnn = self.X_test.reshape((self.X_test.shape[0], self.X_test.shape[1]

    def build_and_train_mlp(self):

        self.mlp_model = Sequential([
            Dense(self.h, activation='relu', input_shape=(self.X_train.shape[1],)),
            Dense(self.h, activation='relu'),
            Dense(2, activation='softmax')])

        self.mlp_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
        self.mlp_model.fit(self.X_train, self.y_train, epochs=self.epochs, batch_size=sel



    def evaluate_mlp(self):
        mlp_loss, mlp_accuracy = self.mlp_model.evaluate(self.X_test, self.y_test)
        print(f'MLP Accuracy: {mlp_accuracy}')
        return mlp_accuracy



class ClassicalML:
    def __init__(self, X_train, X_test, y_train, y_test):
        self.X_train, self.X_test, self.y_train, self.y_test = X_train, X_test, y_train,

    def random_forest(self):
```

```
        rf = RandomForestClassifier(random_state=42)
        rf.fit(self.X_train, self.y_train)
        yt_pred = rf.predict(self.X_train)
        ys_pred = rf.predict(self.X_test)
        tr_accu = accuracy_score(self.y_train, yt_pred)
        ts_accu = accuracy_score(self.y_test, ys_pred)
        print(f'Random Forest Train Accuracy: {tr_accu}, Test Accuracy: {ts_accu}')
        cm = confusion_matrix(y_test, ys_pred)
        ConfusionMatrixDisplay(confusion_matrix=cm).plot()


    def logistic_regression(self):
        lr = LogisticRegression(solver='lbfgs', random_state=42)
        lr.fit(self.X_train, self.y_train)
        yt_pred = lr.predict(self.X_train)
        ys_pred = lr.predict(self.X_test)
        tr_accu = accuracy_score(self.y_train, yt_pred)
        ts_accu = accuracy_score(self.y_test, ys_pred)
        print(f'Logistic Regression Train Accuracy: {tr_accu}, Test Accuracy: {ts_accu}')
        cm = confusion_matrix(y_test, ys_pred)
        ConfusionMatrixDisplay(confusion_matrix=cm).plot()

    def svm(self):
        svm = SVC(kernel='linear', decision_function_shape='ovr', random_state=42)
        svm.fit(self.X_train, self.y_train)
        yt_pred = svm.predict(self.X_train)
        ys_pred = svm.predict(self.X_test)
        tr_accu = accuracy_score(self.y_train, yt_pred)
        ts_accu = accuracy_score(self.y_test, ys_pred)
        print(f'SVM Train Accuracy: {tr_accu}, Test Accuracy: {ts_accu}')
        cm = confusion_matrix(y_test, ys_pred)
        ConfusionMatrixDisplay(confusion_matrix=cm).plot()

    def gradient_boosting(self):
        gb = GradientBoostingClassifier(random_state=42)
        gb.fit(self.X_train, self.y_train)
        yt_pred = gb.predict(self.X_train)
        ys_pred = gb.predict(self.X_test)
        tr_accu = accuracy_score(self.y_train, yt_pred)
        ts_accu = accuracy_score(self.y_test, ys_pred)
        print(f'Gradient Boosting Train Accuracy: {tr_accu}, Test Accuracy: {ts_accu}')
        cm = confusion_matrix(y_test, ys_pred)
        ConfusionMatrixDisplay(confusion_matrix=cm).plot()
```

Load the data

```
Data = DataBase()
```

Describe the unprocessed data

```
print(Data.X.info())
print('\n\n\n')
```

```
print(Data.X.describe())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8499 entries, 0 to 8498
Data columns (total 20 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   aluminium    8479 non-null   float64
 1   ammonia      8473 non-null   float64
 2   arsenic      8487 non-null   float64
 3   barium       8486 non-null   float64
 4   cadmium      8480 non-null   float64
 5   chloramine   8480 non-null   float64
 6   chromium     8485 non-null   float64
 7   copper       8488 non-null   float64
 8   flouride     8487 non-null   float64
 9   bacteria     8491 non-null   float64
 10  viruses      8480 non-null   float64
 11  lead         8490 non-null   float64
 12  nitrates     8489 non-null   float64
 13  nitrites     8490 non-null   float64
 14  mercury      8482 non-null   float64
 15  perchlorate  8489 non-null   float64
 16  radium       8490 non-null   float64
 17  selenium     8490 non-null   float64
 18  silver       8485 non-null   float64
 19  uranium      8489 non-null   float64
dtypes: float64(20)
memory usage: 1.3 MB
None
```

	aluminium	ammonia	arsenic	barium	cadmium
count	8479.000000	8473.000000	8487.000000	8486.000000	8480.000000
mean	3.368098	39.736295	0.780593	4.954736	0.180527
std	88.281232	753.068801	20.430714	94.621737	3.627481
min	0.000000	-725.031300	0.000000	0.000000	0.000000
25%	0.040000	6.590000	0.030000	0.570000	0.008000
50%	0.080000	14.200000	0.050000	1.270000	0.040000
75%	0.730000	22.160000	0.170000	2.560000	0.070000
max	4322.094710	29165.012540	1018.402584	4654.402208	124.871248

	chloramine	chromium	copper	flouride	bacteria
count	8480.000000	8485.000000	8488.000000	8487.000000	8491.000000
mean	6.062729	0.682991	2.200373	1.538828	0.644710
std	126.130093	14.304709	40.745971	26.787621	13.500325
min	0.000000	0.000000	0.000000	0.000000	-0.930000
25%	0.100000	0.050000	0.100000	0.400633	0.000000
50%	0.655517	0.090000	0.760000	0.770000	0.220000
75%	4.440000	0.470000	1.400000	1.160000	0.616303
max	5936.787130	876.021011	1818.959392	1413.452930	822.970807

	viruses	lead	nitrates	nitrites	mercury
count	8480.000000	8490.000000	8489.000000	8490.000000	8482.000000
mean	0.648710	0.202181	28.937860	2.992913	0.010596
std	13.392303	3.411296	495.910983	55.926535	0.203703
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.002000	0.048000	4.970000	1.000000	0.003000

Check the missing values. If "remove" is set to "True", it removes the missing values. The default value is "False".

```
Data.check_missing_values(remove = False)
```

```
Missing data:
aluminium      20
ammonia        26
arsenic        12
barium         13
cadmium        19
chloramine     19
chromium       14
copper         11
flouride       12
bacteria        8
viruses        19
lead            9
nitrates       10
nitrites        9
mercury        17
perchlorate    10
radium          9
selenium        9
silver         14
uranium        10
dtype: int64
```

Check for duplicated rows and columns in the dataset. If "remove" is set to "True", it removes the duplicated rows and columns.

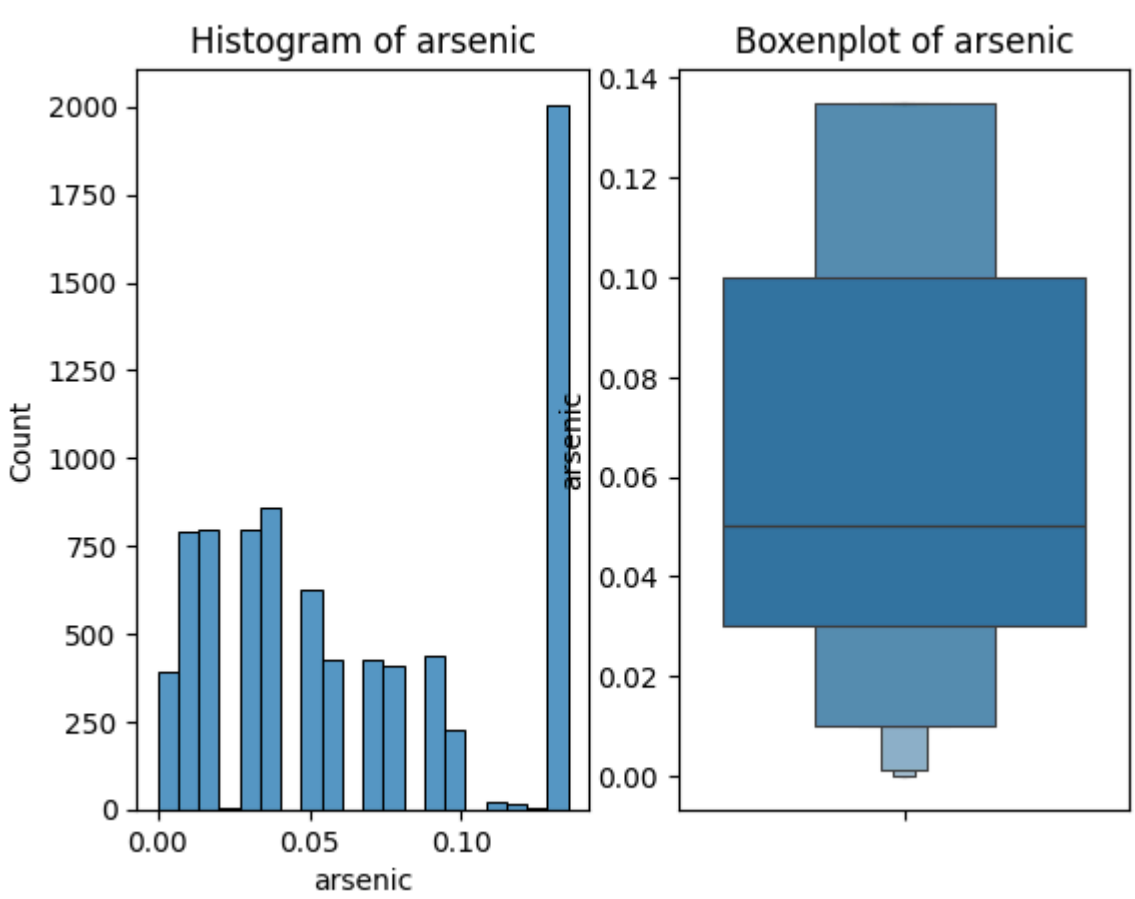
```
Data.check_duplicated_rows(remove = False)
Data.check_duplicated_columns(remove = False)
```

```
No duplicate rows in dataset

No duplicate columns in dataset
```

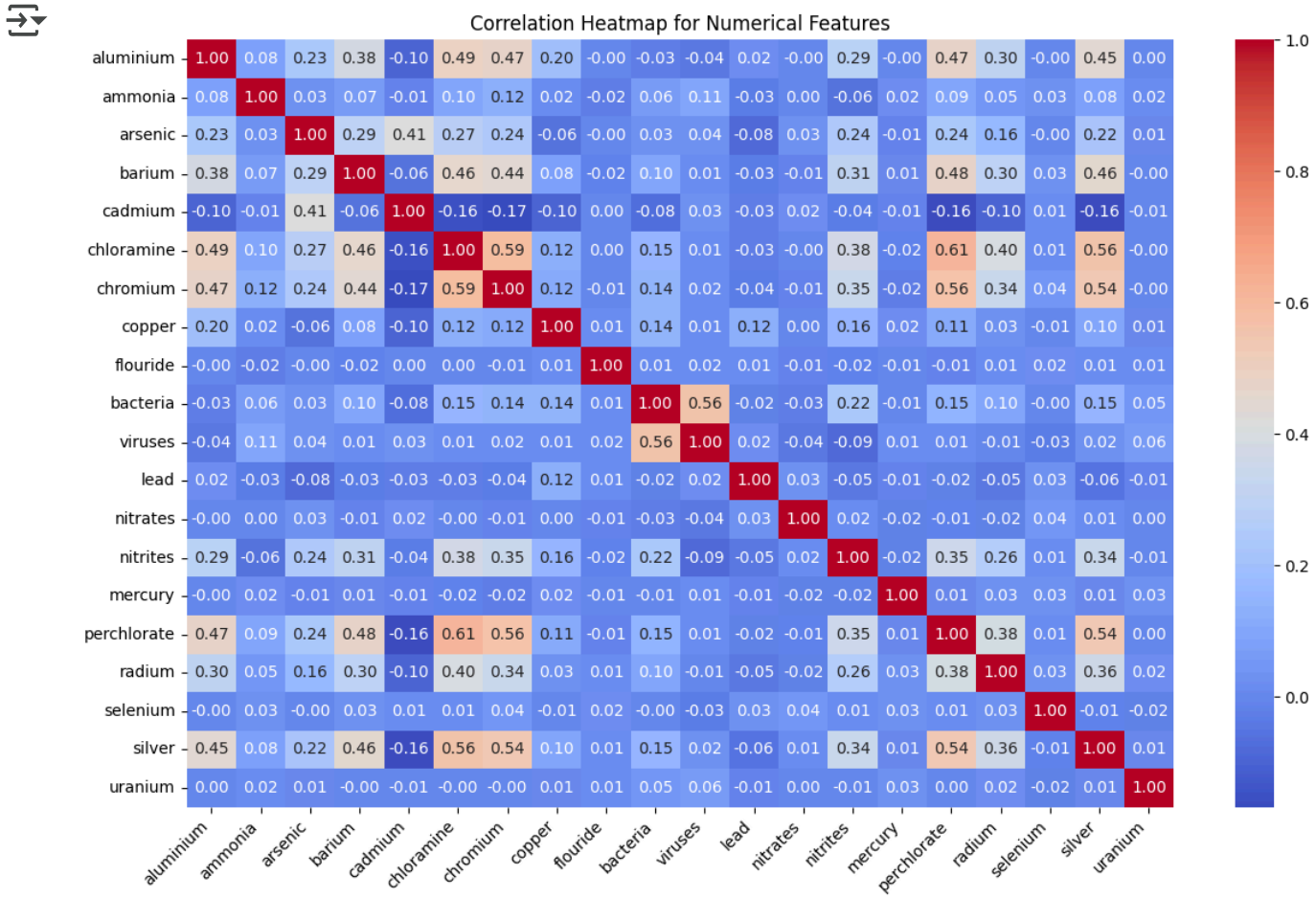
Plot the histogram and boxplot of numerical features (columns). "bins" defines the number of bins for the histogram, with the default set to "auto". "col_index" is the column index. it shold be a number between 0 to 19 (default value for "col_index" is 0). The outlies are removed befor histogram and boxplot ploting

```
col_index = 2 # choose between 0 to 19
Data.histbox_plot(col_index, bins = 'auto')
```



Plot the feature correlations.

```
Data.var_relation_plot()
```



Prepare the X (features) and y (labels). If outlier is set to "True", it removes the outliers, and if scale is set to "True", it scales the features.

```
X, y = Data.prep_data(outlier = True, scale = True)
```

More explanation

```
print(f'Features (columns) name:\n{X.columns}')
print('\n\n\n')
print(f'X dimensions\nnumber of rows: {X.shape[0]}\nnumber of columns: {X.shape[1]}')
```

```
Features (columns) name:
Index(['aluminium', 'ammonia', 'arsenic', 'barium', 'cadmium', 'chloramine',
       'chromium', 'copper', 'flouride', 'bacteria', 'viruses', 'lead',
       'nitrates', 'nitrites', 'mercury', 'perchlorate', 'radium', 'selenium',
       'silver', 'uranium'],
      dtype='object')



X dimensions
number of rows: 8229
number of columns: 20
```

Show the importance of each feature based on their ranks. Feature importance is determined by voting across the outputs of six classifiers, including Random Forest, Support Vector Machine (SVM), Gradient Boosting, Logistic Regression, K-Nearest Neighbors (KNN), and Multi-layer Perceptron (MLP). The ranking is calculated based on feature importance scores generated by each classifier.

```
features = ImpFeature(X, y)
```

```
/usr/local/lib/python3.10/dist-packages/joblib/externals/loky/backend/fork_exec.py:38
  pid = os.fork()
/usr/local/lib/python3.10/dist-packages/sklearn/neural_network/_multilayer_perceptron
  warnings.warn(
/usr/local/lib/python3.10/dist-packages/sklearn/base.py:458: UserWarning: X has featu
  warnings.warn(
```

Show the rank of each features based on the voting strategy over the six classifires.

```
features.find_repetitive_ind()
```

```
rank 1: ['aluminium']
rank 2: ['perchlorate', 'arsenic']
rank 3: ['bacteria', 'perchlorate']
rank 4: ['arsenic', 'perchlorate']
rank 5: ['silver', 'cadmium']
rank 6: ['ammonia']
rank 7: ['ammonia']
rank 8: ['viruses']
rank 9: ['nitrites', 'uranium']
rank 10: ['viruses', 'copper']
rank 11: []
rank 12: ['nitrates']
rank 13: []
rank 14: ['chromium', 'copper']
rank 15: ['radium', 'mercury']
```

```
rank 16: ['mercury', 'lead']
rank 17: ['barium', 'lead']
rank 18: ['flouride', 'selenium']
rank 19: ['selenium', 'barium']
rank 20: ['mercury', 'flouride']
```

Show the rank of each features regarding the classifire algorithm

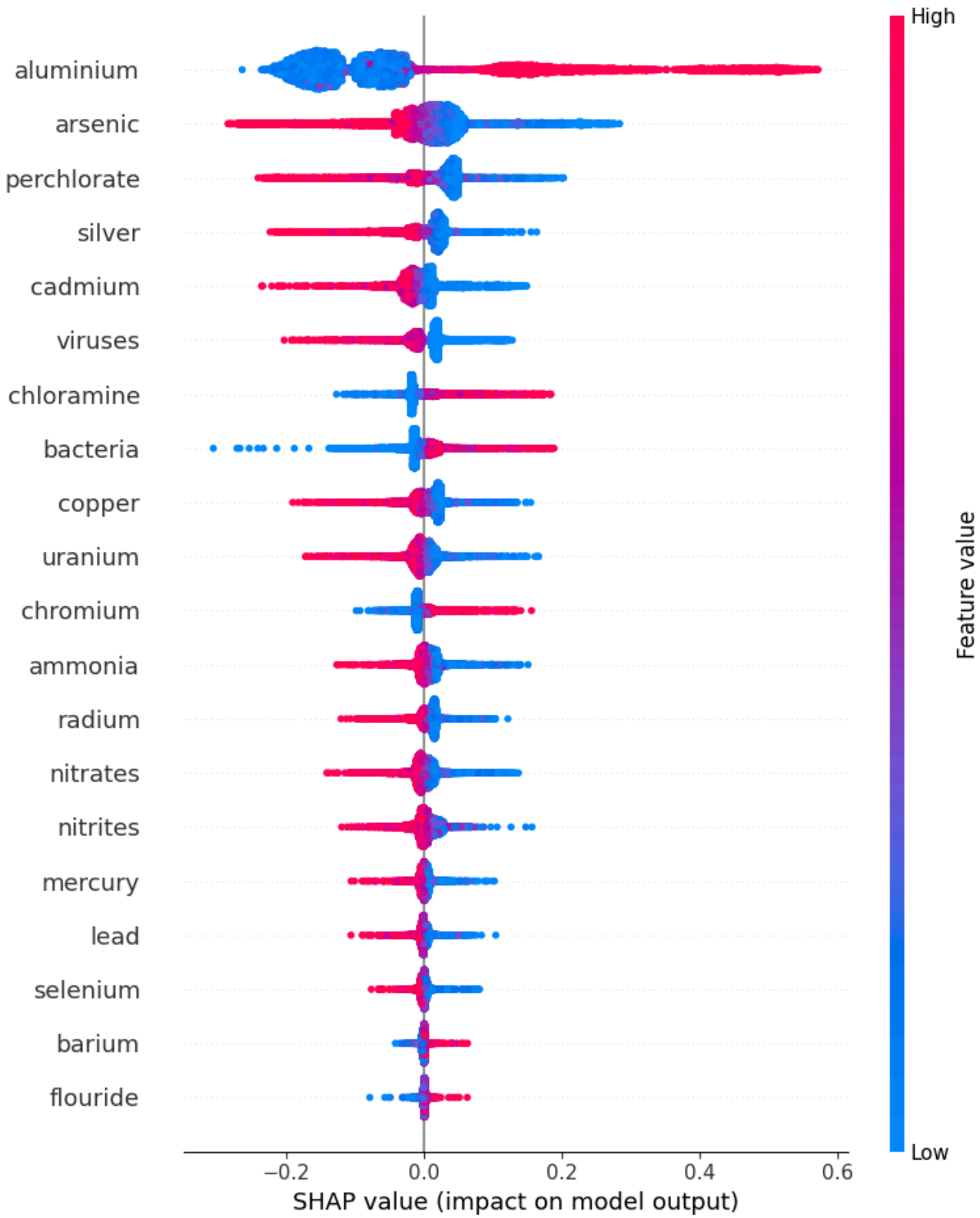
```
name = ['rf', 'svm', 'gb', 'lr', 'knn', 'mlp']
for i in range(len(features.vote)):
  print(f'{name[i]} = {features.vote[i]}')
```

```
rf = [ 0 15  4  2 18  1  5 12 13 10 19 16  6  9 11  7  3  8 17 14]
svm = [ 0  2  9 15  4 18  1 10 19  7 13  5 12  6 16 14 11 17  3  8]
gb = [ 0  4 15  2 18 19  1 13  5 10  9 12 16  7 14 11  3  6 17  8]
lr = [ 0  2  9 15  4  1 19 10 13  7 18 12  5  6 16 14 11 17  3  8]
knn = [ 0  2 15  4 18 12 10  1 19 13  5  3 11 17  9 16  6  8  7 14]
mlp = [ 0 15  2  5 18  4  1 19  6 12 16 10 13  7 14 11 17  8  3  9]
```

Analyze and rank the most important features using SHAP values.

```
features.shap_analysis()
```

100% 8229/8229 [2:43:38<00:00, 1.08s/it]




```
X_train,X_test, y_train, y_test=Data.TrainTest(X,y,tsize=0.3)
```

Build the models for MLP

```
Network = NeuralNetWorkModels(X_train, X_test, y_train, y_test)
```

Train and Evaluate the MLP

```
Network.build_and_train_mlp()
accu = Network.evaluate_mlp()
```

```
Epoch 1/100
/usr/local/lib/python3.10/dist-packages/keras/src/layers/core/dense.py:87: UserWar
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
922/922 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8773 - loss: 0.3211 - val_a
Epoch 2/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9181 - loss: 0.2290 - val_a
Epoch 3/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9140 - loss: 0.2188 - val_a
Epoch 4/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9272 - loss: 0.1838 - val_a
Epoch 5/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9255 - loss: 0.1932 - val_a
Epoch 6/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9375 - loss: 0.1621 - val_a
Epoch 7/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9387 - loss: 0.1641 - val_a
Epoch 8/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9372 - loss: 0.1731 - val_a
Epoch 9/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9404 - loss: 0.1575 - val_a
Epoch 10/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9417 - loss: 0.1546 - val_a
Epoch 11/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9375 - loss: 0.1524 - val_a
Epoch 12/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9382 - loss: 0.1521 - val_a
Epoch 13/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9429 - loss: 0.1507 - val_a
Epoch 14/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9447 - loss: 0.1271 - val_a
Epoch 15/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9415 - loss: 0.1340 - val_a
Epoch 16/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9503 - loss: 0.1352 - val_a
Epoch 17/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9519 - loss: 0.1245 - val_a
Epoch 18/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9540 - loss: 0.1173 - val_a
Epoch 19/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9509 - loss: 0.1173 - val_a
Epoch 20/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9579 - loss: 0.1082 - val_a
Epoch 21/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9592 - loss: 0.1091 - val_a
Epoch 22/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9517 - loss: 0.1157 - val_a
```

```
Epoch 23/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9570 - loss: 0.1053 - val_a
Epoch 24/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9574 - loss: 0.1036 - val_a
Epoch 25/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9564 - loss: 0.1062 - val_a
Epoch 26/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9607 - loss: 0.0955 - val_a
Epoch 27/100
922/922 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9608 - loss: 0.1021 - val_a
Epoch 28/100
```

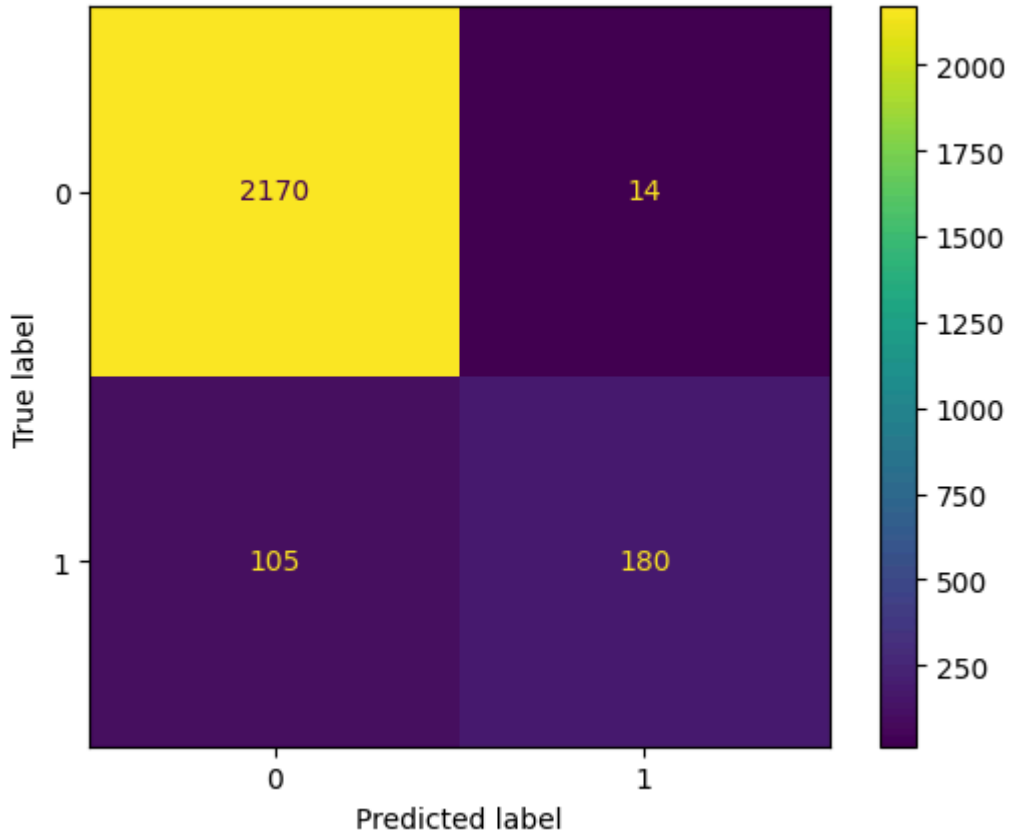
Build the base model for classical ML models, including RF, LR, SVM, and GB.

```
ML = ClassicalML(X_train, X_test, y_train, y_test)
```

RF Output

```
ML.random_forest()
```

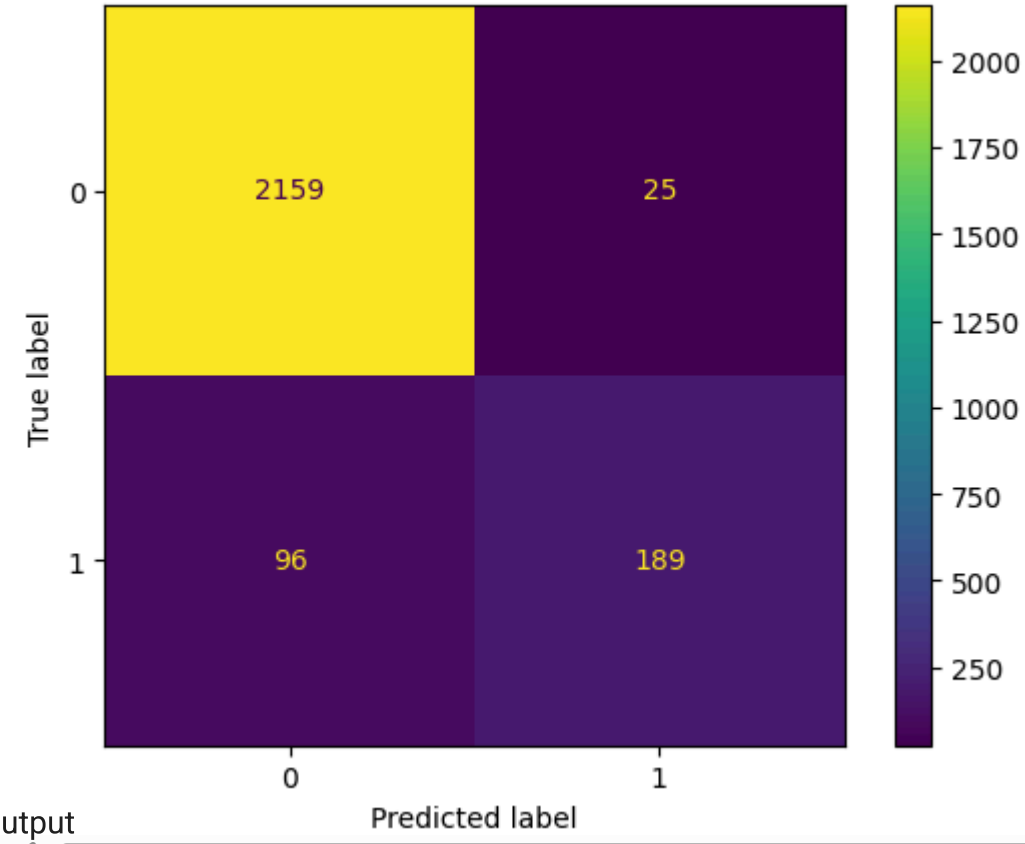
```
Random Forest Train Accuracy: 1.0, Test Accuracy: 0.9518023491292021
```



GB Output

```
ML.gradient_boosting()
```

Gradient Boosting Train Accuracy: 0.9571180555555555, Test Accuracy: 0.95099230457675



LR Output

```
ML.svm()
```

SVM Train Accuracy: 0.9170138888888889, Test Accuracy: 0.9210206561360875